Desarrollo de una aplicación web como TFG

1 Introducción 1

1.1 Temática de la aplicación web 1

1.2 Tecnologías y arquitectura de la aplicación web 3

1.3 Partes optativas 3

2 Proceso de desarrollo del TFG 6

2.1 Fases del desarrollo 6

2.2 Seguimiento del tutor 7

2.3 Metodología software 8

2.4 Herramientas colaborativas para el desarrollo 8

2.5 Código de calidad 9

2.6 Documentación 9

3 Fase 1: Definición de funcionalidades y pantallas 10

3.1 Diseño de las pantallas y navegación 12

3.2 Documentación 13

3.3 Material de apoyo 15

4 Fase 2: Repositorio, pruebas y CI 15

4.1 Funcionalidad mínima 16

4.2 Pruebas automáticas 16

4.3 Integración Continua 17

4.4 Documentación 18

4.5 Material de apoyo 21

5 Fase 3 - Versión 0.1 - Funcionalidad básica y Docker 22

5.1 Backend 22

5.2 Frontend 23

5.3 Controles de calidad 24

5.4 Empaquetado con Docker y Docker Compose 25

5.5 Entrega continua y publicación de versiones 26

5.6 Documentación 27
6 Fase 4 - Versión 0.2 - Funcionalidad intermedia 30
6.1 Despliegue 31
6.2 Documentación 31
7 Fase 5 - Versión 1.0 - Funcionalidad avanzada 31
8 Fase 6: Memoria 32
9 Fase 7: Defensa 32
10 TFGs para alumnos con doble titulación 32

1 Introducción

Este documento describe las características de un Trabajo de Fin de Grado de tipo Web tutorizado por los profesores de asignaturas relacionadas con la web en la ETSII (Michel Maes, Óscar Soto, Iván Chicano o Micael Gallego).

Los TFGs se publicarán con licencia libre Apache 2 y su memoria con licencia libre Creative Commons.

De forma resumida, las características de este tipo de TFGs son las siguientes:

- **Temática de la aplicación web:** Podrá ser elegida libremente por el alumno.
- **Tecnologías y arquitectura de la aplicación web:** Se realizará una aplicación con arquitectura SPA, con pruebas automáticas, CI/CD, empaquetado con Docker y despliegue. El alumno podrá elegir las tecnologías concretas utilizadas. Además, deberá elegir entre un conjunto de partes optativas como son las mejoras en el control de calidad, despliegue en plataformas avanzadas, distribución del backend, pruebas con usuarios, etc.
- **Metodología de desarrollo:** El trabajo se realizará en las siguientes fases a lo largo de un curso académico.

Fase	Descripción
Fase 1	Definición de funcionalidades y pantallas
Fase 2	Repositorio, pruebas y CI
Fase 3	Versión 0.1 - Funcionalidad básica y Docker
Fase 4	Versión 0.2 - Funcionalidad intermedia
Fase 5	Versión 1.0 - Funcionalidad avanzada
Fase 6	Memoria
Fase 7	Defensa

A continuación, se describen estos puntos en mayor detalle.

1.1 Temática de la aplicación web

La temática de la aplicación web que hay que diseñar e implementar será elegida libremente por el alumno. Decidirá qué funcionalidades ofrece al usuario, el aspecto gráfico, el esquema de navegación, etc.

A modo de ejemplo, se presentan algunos tipos de aplicaciones web que se podrían implementar, pero el alumno podrá elegir cualquier temática (aunque no esté aquí listada):

- Web de compra/venta de objetos usados (listado, proceso de compra, registro de compras, comunicación entre comprador y vendedor).
- Web de selección/contratación de cuidadores de niños por horas (listado, valoraciones/reputación, filtrado por disponibilidad, características...).
- Web de un gimnasio con seguimiento de entrenamiento (clases colectivas, horarios, registro, información pública...).
- Web de una liga de fútbol / torneo de pádel (equipos, jugadores, partidos, clasificación, calendario...).
- Web de gestión docente de una universidad (horarios, fechas de exámenes, asignación de profesores a asignaturas, fichas de asignaturas, alumnos...).
- Web de un ayuntamiento (noticias, votaciones, registrarse en actividades, calendario de actividades...).
- Web para diseñar viajes turísticos (lugares de interés, planificación, mapas, horarios...).
- Web de una academia de formación (cursos ofertados, activos, reserva, matrícula...).
- Web de venta online (productos, carrito de la compra, stock, pedido, análisis de ventas...).
- Web de reserva de aulas de informática en la universidad (registro de software por aula, calendario, conflictos, gestión manual...).
- Web para formación (tipo Moodle) (asignaturas/cursos por alumno, material trabajo, entrega de trabajos, foro...).
- Web para seguimiento y evaluación de una práctica por fases: (equipos, notas del equipo y de cada integrante, lista de ítems de corrección, análisis de datos, fechas...).

Es posible copiar alguna web que esté disponible en Internet. Si copia una web existente, se podrán copiar su diseño gráfico, logotipos, iconos, etc.

La elección de la funcionalidad de la web es importante, ya que se considera al evaluar el trabajo realizado en la sección “Descripción del contexto y del problema abordado” de la rúbrica de evaluación.

1.2 Tecnologías y arquitectura de la aplicación web

Una vez elegida la temática de la aplicación web, se deberá seleccionar con qué tecnologías se implementa, qué arquitectura tiene y otros aspectos complementarios.

Las tecnologías recomendadas son las que se imparten en las asignaturas de la ETSII:

- **Backend**: API REST en Java y Spring Boot
- **Frontend**: Angular (SPA)
- **Base de datos**: MySQL
- **Repositorio**: GitHub
- **CI/CD**: GitHub Actions
- **Pruebas automáticas**: Java, Junit, Selenium y Rest Assured

El alumno podrá elegir otras tecnologías alternativas, pero no se tendrá el soporte técnico del profesor:

- **Backend**: Se podrá cambiar Spring por Quarkus, Micronaut, JavaScript / TypeScript con Node (express), Python (Django, Flask, ...), C# (ASP.NET) etc.
- **Frontend**: Se podrá cambiar Angular por otras tecnologías de desarrollo de aplicaciones SPA como Vue.js, React, Svelt, etc. o tecnologías de desarrollo de Apps móviles nativas o híbridas.
- **Base de datos**: Se podrá cambiar MySQL por PostgreSQL, opciones NoSQL como MongoDB, etc.
- **Pruebas automáticas**: Se podrá cambiar JUnit y Selenium por tecnologías similares como Jest, Playwright, etc. Incluso se podrán implementar con otros lenguajes de programación diferentes a Java como Python, JavaScript / TypeScript, etc.
- **Repositorio**: GitLab, AWS Codecommit, etc.
- **CI/CD**: Se podrá utilizar Jenkins, Travis, etc.

En principio, la aplicación se implementará siguiendo las siguientes características arquitectónicas:

- **Backend**: Monolito con API REST
- **Frontend**: SPA comunicada con API REST
- **Empaquetado**: Docker

1.3 Partes optativas

Además de las partes fijas identificadas para el desarrollo del TFG, el alumno tendrá que seleccionar entre algunas partes optativas. Hay partes que afectan a la arquitectura de la aplicación, a aspectos tecnológicos o bien a otras tareas del proceso de desarrollo.

El alumno tendrá que elegir entre las siguientes partes optativas con un mínimo de 3 puntos. Para cada punto se aporta material de apoyo relevante si lo tenemos disponible:

- **Pruebas automáticas unitarias y de integración (2 puntos):** Añadir pruebas unitarias y de integración en backend y frontend.
    - Se empezarán en la Fase 2 e irán evolucionando en el resto de fases.
    - En el backend las pruebas mínimas serán:
        - Unitarias: Lógica de negocio de los servicios.
        - Integración: Servicios contra una base de datos real.
    - En el frontend las pruebas mínimas serán:
        - Unitarias: Verificar que el comportamiento de los componentes considerando un doble de la API REST.
        - Integración: Verificar el comportamiento de los servicios considerando la API REST real.
    - Material relevante: pruebas
- **Análisis estático de código (1 punto):** Uso de herramientas de análisis estático de código en la nube (p.e. Sonar) y revisión de las violaciones reportadas.
    - Se configurará en la Fase 2 y se realizará el seguimiento durante todo el desarrollo.
    - Material relevante: calidad
- **Empaquetado nativo (0.5 puntos):** Generación de imagen nativa con GraalVM.
    - Se realizará en la fase 2 porque el empaquetado nativo impide usar ciertas funcionalidades de Spring y Java. Por tanto, es importante que se sepa cuanto antes.
- **Plataforma de despliegue.** Material relevante: cloud (AWS):
    - Despliegue básico en máquina virtual en la nube (IaaS) (0.5 puntos)
        - Se realizará en la Fase 4
    - Despliegue básico en máquina virtual en la nube con base de datos gestionada (1 puntos)
        - Se realizará en la Fase 5
    - Despliegue básico en plataforma en la nube (PaaS) (1 punto): Azure App Service, Heroku, Railway, Beanstalk, etc)
        - Se realizará en la Fase 5
    - Despliegue con Kubernetes en la nube (2 puntos). Material relevante: contenedores (Kubernetes)
        - Se realizará en la Fase 5
    - Despliegue integrado con servicios Cloud (S3, RDS, Cloud Formation…) (3 puntos)
        - Se realizará en la Fase 5 a menos que el número de servicios sea numeroso y muchas funcionalidades de la aplicación los requieran, entonces se empezará en la Fase 4.
- **Despliegue continuo (1 punto).** Material relevante: devops
    - Se realizará en la Fase 5.

- **Arquitectura distribuida:**
  - Dividir en dos servicios (1 punto)
    - Se iniciará en la Fase 5 si uno de los servicios se puede considerar "complementario" y no se usa hasta la Fase 4.
    - Si la funcionalidad del servicio se considera básica, se iniciará en la Fase 3.
  - Dividir en tres o más servicios (2 puntos)
    - Se iniciará en la Fase 3.
  - Utilizar tecnologías de comunicación complementarias a REST (1 punto): gRPC, GraphQL, mensajería con RabbitMQ, Kafka, WebSockets, etc.
    - Se iniciará en la Fase 5 si uno de los servicios se puede considerar "complementario" y no se usa hasta la Fase 5.
    - Si la funcionalidad del servicio se considera básica, se iniciará en la Fase 3.
    - Material relevante: tecnologias-comunicacion
  - Escalabilidad y tolerancia a fallos: Escalado horizontal, pruebas de carga y tolerancia a fallos (3 puntos).
    - Se iniciará en la Fase 4
    - Material relevante: escalabilidad
- **Diseño responsive en móvil (1 punto)**
  - Se iniciará en la Fase 3 para que el desarrollo responsive se desarrolle en paralelo con el desarrollo en pantalla.

2 Proceso de desarrollo del TFG

El proyecto deberá realizarse siguiendo la siguiente metodología de desarrollo.

2.1 Fases del desarrollo

El proyecto será desarrollado en diferentes fases. Las fases en las que se divide el desarrollo de la aplicación web y sus fechas límite son:

Fase	Descripción	Fecha límite
Fase 1	Definición de funcionalidades y pantallas	15 septiembre
Fase 2	Repositorio, pruebas y CI	15 octubre
Fase 3	Versión 0.1 - Funcionalidad básica y Docker	15 diciembre
Fase 4	Versión 0.2 - Funcionalidad intermedia	1 marzo
Fase 5	Versión 1.0 - Funcionalidad avanzada	15 abril
Fase 6	Memoria	15 mayo
Fase 7	Defensa	15 de junio

A continuación, se describen estas fases en un poco mayor nivel de detalle.

- **Fase 1 - Definición de funcionalidades y pantallas:** En esta fase se definirá la funcionalidad de la web, el diseño de interacción (pantallas, transiciones, etc). Se diferenciará entre la funcionalidad para diferentes roles de usuario (usuario no registrado, usuario registrado y administrador). Estas funcionalidades se dividirán en:
    - **Funcionalidad básica:** Funcionalidad básica de los usuarios no registrados, registrados y administradores.
    - **Funcionalidad intermedia:** Funcionalidad adicional.
    - **Funcionalidad avanzada:** Funcionalidad final.
- **Fase 2 - Repositorio, pruebas y CI:** Se creará el repositorio git, los proyectos de cliente y servidor y se implementará la funcionalidad mínima para conectar cliente, servidor y base de datos. Se implementarán unos mínimos tests automáticos y se configurará el sistema de CI.
- **Fase 3 - Versión 0.1 - Funcionalidad básica y Docker:** Se ampliará la funcionalidad hasta la funcionalidad básica (con sus correspondientes pruebas automáticas) y se empaquetará la aplicación en Docker. Se añadirá la capacidad de entrega continua. Se publicará la versión 0.1 de la aplicación.
- **Fase 4 - Versión 0.2 - Funcionalidad intermedia:** Se ampliará a la funcionalidad intermedia (con sus correspondientes pruebas automáticas) y se publicará la versión 0.2 de la aplicación. En esta fase también se desplegará la aplicación.
- **Fase 5 - Versión 1.0 - Funcionalidad avanzada:** Se finalizará la aplicación y se publicará la versión 1.0.
- **Fase 6 - Memoria:** Se elaborará el primer borrador de la memoria.
- **Fase 7 - Defensa:** Se realizará el acto de defensa del TFG.

El contenido de cada una de estas fases se describe en detalle en las siguientes secciones del documento.

Nota: Las fechas límite para la realización de cada una de las fases no podrán superarse (salvo causa de fuerza mayor). Los alumnos deben comprometerse a realizar el TFG en un curso académico, aunque comiencen a trabajar. Si un alumno incumple los plazos tendrá que buscar otro tutor y realizar el TFG en otra temática.

2.2 Seguimiento del tutor

El tutor actuará tendrá un rol similar al jefe de proyecto, marcando prioridades, haciendo un seguimiento del cumplimiento de la planificación, etc. También actuará como líder técnico (o arquitecto) en aspectos técnicos.

El tutor no realizará una labor de supervisión técnica de bajo nivel, porque se asume que el alumno ya ha adquirido los conocimientos necesarios para realizar el TFG. Y si no los ha adquirido, debería ser autónomo para adquirirlos y solventar los problemas que vaya encontrando. Dicho de otro modo, el tutor no actuará como un profesor que vaya a

resolver cada una de las dudas técnicas que surjan. En caso de que haya problemas técnicos que no se puedan solventar, el profesor propondrá alternativas al alumno para que pueda concluir el trabajo.

El tutor no actuará como cliente o como usuario, ya que el alumno debe definir la funcionalidad de la aplicación web. No obstante, si podrá proponer cambios para que la aplicación cumpla con unos mínimos de usabilidad y tamaño.

El seguimiento del trabajo se realizará sobre todo usando el correo electrónico, chat y videoconferencias de Teams y el proyecto de GitHub. Se tendrán reuniones periódicas para supervisar que la planificación se va cumpliendo. El alumno podrá solicitar reuniones bajo demanda.

Para facilitar la supervisión, el alumno publicará en un blog en medium los avances que vaya haciendo en el proyecto. Ese blog servirá como promoción de su trabajo. Cuando el alumno publique una entrada enviará un mail al profesor para que esté al tanto de la misma. Este blog se puede escribir en inglés o castellano, aunque se recomienda que sea en inglés para que el alumno practique este lenguaje.

A continuación, se muestran algunos blogs de compañeros que ya han realizado o están realizando su TFG:

- https://medium.com/@izanrb
- https://medium.com/@ivchicano
- http://full-teaching.blogspot.com.es/
- https://ejimenezgrande.wordpress.com
- http://desarrollandomitfg.blogspot.com.es
- https://itsnotafunction.wordpress.com

2.3 Metodología software

El proyecto se desarrollará siguiendo una metodología iterativa e incremental. Se realizarán varias iteraciones. En cada una de esas iteraciones se implementarán una serie de requisitos funcionales y se realizarán una serie de tareas técnicas.

Es importante registrar de forma adecuada el tiempo dedicado a cada una de las fases que se siguen en el desarrollo del proyecto y las tareas incluidas en cada fase. Con esta información se generará un diagrama de tipo Gantt en el que se reflejarán las fechas de inicio y fin de cada fase y tarea dentro de esa fase. Además, se registrarán las horas reales dedicadas a cada fase, pues habrá periodos en los que se pueda dedicar más tiempo o menos al TFG.

Hay que tener en cuenta que la metodología es uno de los aspectos que son valorados en la rúbrica de evaluación del TFG.

2.4 Herramientas colaborativas para el desarrollo

2.4.1 GitHub

La aplicación web se desarrollará usando un repositorio de la plataforma GitHub[1]. El repositorio será creado por el tutor.

Se usará GitHub Flow[2] para la gestión de ramas:

- Rama *main/master:* Rama estable del nuestro proyecto. Nunca se hace commits directos sobre ella. El código de esta rama está listo para desplegar.
- Ramas *feature/fix*: Serán las ramas dónde se desarrollarán las funcionalidades o las correcciones (fixes). Deberán tener un nombre corto y descriptivo en inglés (p.e. *add-login-page o fix-login-page*).

La estrategia de integración de cambios de las ramas *feature* a la rama *main* será a través de pull-request.

Los mensajes de *commit* también tienen que ser adecuados y describir correctamente el objetivo del *commit*[3] [4].

Para gestionar el repositorio de Github se puede usar cualquier cliente de git.

2.4.2 GitHub Issues

La gestión de las tareas del proyecto se realizará con Issues de Github[5].

El alumno deberá crear diferentes tareas por cada una de las fases reflejadas en este proceso de desarrollo y cambiar su estado a media que las va completando. El tutor revisará esta herramienta para analizar el avance del desarrollo del TFG.

En concreto, se usará GitHub projects[6] para gestionar el trabajo con una vista en forma de tablero Kanban[7].

2.4.3 OneDrive

Además, se utilizará una carpeta compartida en OneDrive con un documento de seguimiento del TFG en el que se tomarán notas de las reuniones, se podrá compartir

---

[1] https://github.com/

[2] https://github.com/trein/dev-best-practices/wiki/Git-Commit-Best-Practices

[3] https://github.com/erlang/otp/wiki/writing-good-commit-messages

[4] https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects

[5] https://github.com/features/issues

[6] https://docs.github.com/en/issues/planning-and-tracking-with-projects

[7] https://docs.github.com/en/issues/planning-and-tracking-with-projects/customizing-views-in-your-project/changing-the-layout-of-a-view#about-the-board-layout

documentación, etc. Este documento será creado por el profesor y compartido con el alumno en la primera reunión de coordinación.

2.5 Código de calidad

El proyecto se deberá realizar cumpliendo altos estándares de calidad:

- Pruebas automáticas
- El código debe seguir las reglas de estilo
- El software debe tener buen diseño y ser mantenible (modular, extensible, etc.)

2.6 Documentación

La documentación del proyecto se incluirá en el fichero *README.md* en la raíz del repositorio de código. Este fichero se editará usando adecuadamente el formato *Markdown* (títulos, negritas/cursivas, tablas, texto en formato código, etc). Se debe verificar que el documento *README.md* se visualiza correctamente desde la web de GitHub.

En la descripción de cada una de las fases se indica de forma detallada el contenido de la documentación solicitada.

3 Fase 1: Definición de funcionalidades y pantallas

En la fase 1 se deben definir las funcionalidades de la web.

Para la definición de la funcionalidad hay que estudiar otras aplicaciones que ofrezcan una funcionalidad similar para obtener ideas y posibles mejoras. Este estudio será incluido en la sección de “Estado del arte” de la memoria.

Las funcionalidades se deberán separar por prioridad:

- **Funcionalidad básica:** Funcionalidad básica de los usuarios no registrados, registrados y administradores.
- **Funcionalidad intermedia:** Funcionalidad adicional.
- **Funcionalidad avanzada:** Funcionalidad que permite obtener la versión final. Son aquellas funcionalidades que están menos detalladas, son más exploratorias o que suponen la integración con otros servicios (servicios en la nube, APIs REST, etc..) o con nuevas tecnologías (de visualización, de búsqueda, IA, etc...).

La funcionalidad de la aplicación web se deberá diseñar de forma que cumpla con las siguientes características:

- **Entidades:** Una entidad representa un concepto que la aplicación web guarda en la base de datos. Son las clases de dominio, las tablas de la base de datos. La aplicación web deberá gestionar, al menos 4 entidades. Una de ellas será la

entidad Usuario (para guardar la información de los usuarios que acceden a la web). Las otras 3 entidades dependen de la temática de la web. Es importante que las entidades estén relacionadas entre sí. Por ejemplo:

  - **Web de gestión de torneos o ligas:** Entidades usuario, equipo, torneo y partido. La relación se tiene porque un equipo juega en partidos que forman parte de un torneo.
  - **Web de gestión de cursos:** Entidades usuario, curso, material, mensaje. La relación se tiene porque un alumno está en un curso, que tiene materiales y mensajes en el foro.
  - **Web de cuidado de niños:** Entidades usuario (progenitor o cuidador), solicitud, jornada de cuidado. La relación se tiene porque una jornada de trabajo ha sido realizada a un progenitor por un cuidador. Además, el progenitor puede poner una solicitud, que se puede convertir en una jornada realizada por un cuidador.
  - **Web de gestión de exámenes:** usuario (profesor o alumno), examen, asignatura, realización de examen. Las entidades están relacionadas porque un profesor imparte una asignatura con alumnos. Esos alumnos tendrán que realizar el examen de la asignatura.

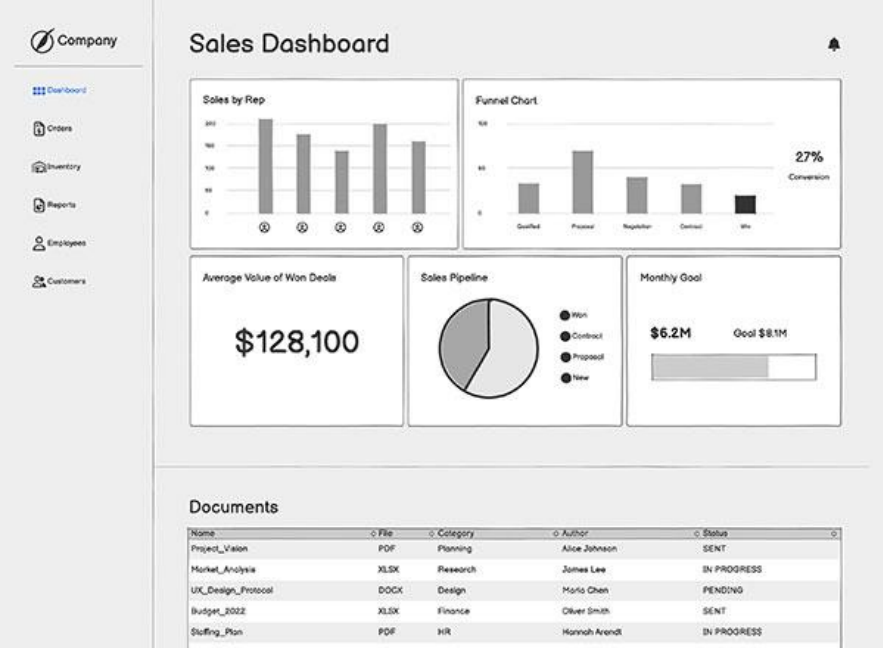
- **Tipos de usuarios:** La aplicación web deberá considerar tres tipos de usuarios.
  - **Usuario anónimo:** Aquel usuario que visita la web y no introduce ningún tipo de credenciales para consultar contenido y realizar búsquedas. Salvo que esté justificado por el tipo de aplicación, este usuario sólo podrá consultar información de la web, pero no podrá crearla ni modificarla.
  - **Usuario registrado:** Aquel usuario que tiene que usar sus credenciales para acceder a la web. Este usuario tendrá datos personalizados como su nombre, una imagen, el histórico de acciones realizadas en la web (mensajes en foros, compras, etc..). La web deberá permitir el registro de nuevos usuarios.
  - **Usuario administrador:** Aquel usuario que tiene control total sobre la información de la web. Por ejemplo, el alta de productos, la creación de los campeonatos, registro de información o actividades, etc. La web tendrá un único usuario administrador con una contraseña especificada en un fichero de configuración (cifrada).
- **Permisos de los usuarios:** La web tiene que estar diseñada para que los usuarios registrados puedan ser dueños de ciertos datos. Por ejemplo, si se trata de una tienda, los pedidos previos que ha realizado. Si es una web de contratación de cuidadores, el histórico de cuidados. Si es una web de un gimnasio, los comentarios que pone a cada clase. Esta funcionalidad es importante porque la web debe implementar los mecanismos de seguridad adecuados para que sólo el usuario que ha creado un elemento (su dueño) pueda borrarlo o editarlo.

- **Imágenes:** La web tiene que permitir la subida de imágenes desde el navegador web. Por ejemplo, como avatar de los usuarios, fotos de productos, etc.
- **Gráficos:** Se deberán usar gráficos (charts) para mostrar algún tipo de información en la web.
    - **Web de gestión de torneos o ligas:** Gráfica de líneas con la puntuación/clasificación de los equipos a lo largo del tiempo.
    - **Web de gestión de cursos:** Número de alumnos registrados en cada uno de los cursos.
    - **Web de cuidado de niños:** Gráfica de líneas con las jornadas realizadas por mes para un cuidador.
    - **Web de gestión de exámenes:** Gráfica de barras con el número de alumnos por asignatura.
- **Tecnología complementaria:** Se deberá hacer uso de alguna funcionalidad que se implemente con alguna tecnología o librería:
    - Envío de correos a los usuarios.
    - Generación de PDFs. (facturas, entradas, etc.)
    - Uso de websockets para implementar edición o avisos en tiempo real (chat, pizarra compartida, etc.)
    - Uso de mapas de GoogleMaps / OpenStreetMap para posicionar elementos de la aplicación web (restaurantes, pedidos, etc.)
    - Uso de una API REST de algún servicio externo (Información meteorológica, libros, películas, etc).
- **Algoritmo o consulta avanzada:** La aplicación web deberá ofrecer alguna funcionalidad que requiera la implementación de un algoritmo o un tratamiento avanzado sobre los datos que gestiona. No basta con que las entidades se puedan crear, modificar, actualizar y borrar.
    - Si se opta por una web que permita hacer el seguimiento de una liga de fútbol, la clasificación se deberá calcular de forma automática a medida que se vayan registrando los resultados los partidos.
    - Si se opta por una web de contratación de cuidadores para niños, se implementará un sistema de valoraciones, de forma que la búsqueda tenga en cuenta las valoraciones (mejor valorados primero), o una búsqueda basada en la distancia a la que puede ir el cuidador, etc.
    - Si es una página web de venta de productos, se implementará un sistema de ofertas personalizadas en base a los productos comprados previamente por el usuario (por ejemplo, mostrar como recomendado un producto de la categoría que más compró el usuario en el pasado).
    - Si se diseña una página de gestión de exámenes, se deberán poder analizar las posibles restricciones que existan (que un mismo alumno no tenga dos exámenes el mismo día, que un profesor no tenga exámenes en el mismo momento, etc).

3.1 Diseño de las pantallas y navegación

Además de las funcionalidades de la web, se debe realizar el diseño de las pantallas (páginas) y la navegación entre ellas. En este diseño de pantallas se puede dejar fuera la funcionalidad avanzada, ya que generalmente es la menos se detalla porque requiere la investigación de tecnologías o servicios.

Para realizar el diseño no se recomienda usar HTML y CSS, sino creando bocetos o esquemas, conocidos como "wireframe". Para ello se puede usar boli y papel (y luego haciendo fotos para digitalizarlo) o bien usando herramientas de diseño específicamente diseñadas para ello (por ejemplo figma, lucidchart, etc.). En la siguiente figura se muestra un ejemplo de wireframe.



El objetivo de esta técnica es que se diseñe la interacción de la página con un coste bastante bajo (frente a la maquetación con HTML y CSS) y el profesor lo pueda evaluar y sugerir los cambios que fueran necesarios por motivos de usabilidad o complejidad. De esa forma, se puede realizar la maquetación y el desarrollo de las funcionalidades sobre una base validada.

3.2 Documentación

Se creará un fichero *README.md* usando el formato *Markdown* en la raíz del repositorio de GitHub. Este fichero se visualizará al entrar en la web de GitHub de ese repositorio y tendrá que estar correctamente formateado usando secciones, subsecciones, tablas, código fuente como texto en formato monoespaciado, imágenes, etc.

La documentación se podrá escribir en castellano, aunque se recomienza que se haga en inglés.

El contenido del fichero *README.md* deberá contener la siguiente información:

- **Título: Nombre de la aplicación web**.
- Un párrafo que resuma la funcionalidad que tendrá la aplicación web
- Algunos bocetos de pantalla que ilustren la funcionalidad
- Se debe indicar claramente que sólo se han definido los objetivos funcionales y los objetivos técnicos de la aplicación, pero no se ha comenzado su implementación todavía.
- **Objetivos**
    - Objetivos con los que se aborda esta aplicación:
        - Objetivos funcionales:
            - Un párrafo resumiendo los objetivos funcionales.
            - Una lista de 3-10 funcionalidades detallando un poco más el párrafo de objetivos funcionales.
        - Objetivos técnicos:
            - Un párrafo resumiendo los aspectos tecnológicos planificados.
            - Una lista de 3-10 aspectos técnicos detallando un poco más el párrafo de objetivos técnicos.
- **Metodología**
    - Se describirá en alto nivel cómo se va a desarrollar el trabajo.
    - Fases:
        - Fase 1: Definición de funcionalidades. Se indicará las secciones concretas en la que se describe la funcionalidad general y detallada.
        - Fase 2: Configuración de las tecnologías y herramientas de desarrollo con controles de calidad que se realizan de forma periódica.
        - Fases 3, 4, 5: Desarrollo iterativo e incremental de la aplicación. Al final de cada fase se publicará una versión (release).
        - Fase 6: Escritura de la memoria.
        - Fase 7: Preparación de la presentación.
    - Se incluirán las fechas de inicio y fin de cada fase.
    - Se elaborará un diagrama de Gantt en el que se muestren gráficamente estas fases.
- **Funcionalidades detalladas**
    - Se deben identificar las consideradas básicas, intermedias y avanzadas.
    - Se debe definir claramente a qué tipo de usuario van dirigidas.
    - Se deben separar por básicas, intermedias y avanzadas.
- **Análisis**
    - **Pantallas y navegación:** Se mostrará el mockup de cada pantalla, una breve descripción de cada una y a las páginas a las que se accede desde ella.

    - **Entidades:** Con sus atributos principales y sus relaciones.
    - **Permisos de usuarios:** Describir los permisos de cada tipo de usuario.
    - **Imágenes:** Indicar qué entidades tendrán asociadas uno o varias imágenes por cada elemento.
    - **Gráficos:** Qué información se mostrará usando gráficos. De qué tipo serán los gráficos (líneas, barras, tarta, etc).
    - **Tecnología complementaria:** Qué tecnología complementaria se empleará.
    - **Algoritmo o consulta avanzada:** Cuál será el algoritmo o la consulta avanzada que se implementará.
- **Seguimiento**
    - Blog con anuncios sobre el desarrollo del proyecto
    - GitHub Project usado para gestionar las tareas del proyecto
- **Autor**
    - Introducción indicando que el desarrollo de esta aplicación se hace en el contexto del Trabajo de Fin de Grado de la titulación X en la ETSII de la URJC.
    - Nombre del alumno y del tutor.

3.3 Material de apoyo

- Git: git

4 Fase 2: Repositorio, pruebas y CI

Esta fase tiene como objetivo configurar todas las herramientas y tecnologías en el sistema local y de integración continua para poder realizar el desarrollo de forma profesional, siguiendo unos estándares de calidad desde el principio.

En concreto, se realizarán las siguientes tareas:

- Se clonará el repositorio GitHub creado por el tutor para el desarrollo del TFG.
- Se creará el proyecto para el desarrollo del servidor con Spring (o la tecnología seleccionada) en la carpeta “backend”.
- Se creará el proyecto para el desarrollo del cliente con Angular (o la tecnología seleccionada) en la carpeta "frontend”.
- Se implementará una funcionalidad mínima para mostrar en el cliente los datos básicos de la entidad principal guardada en la base de datos (esto garantiza la correcta conexión entre todas las partes de la aplicación).
- Se implementarán las pruebas automáticas que verifican que los datos se muestran correctamente.

- Se configurará el sistema de CI para que se realice el control de calidad básico en cada commit y un control de calidad más completo cuando se pretenda actualizar la rama main (con un merge desde la rama de funcionalidad).

Si se han seleccionado algunas partes optativas que se deban realizar en esta fase, también deberán tenerse en cuenta.

4.1 Funcionalidad mínima

La funcionalidad mínima permite ejecutar la aplicación web y verificar que cada una de sus partes se integran correctamente (cliente, servidor y base de datos).

La funcionalidad mínima consiste en:

- Se visualizan en la página inicial unos datos de ejemplo de la entidad principal del modelo de datos.
- No es necesario que la entidad principal tenga todos sus atributos.
- No se incluirán las relaciones con otras entidades.
- Los datos de ejemplo pueden ser de prueba, no tienen que ser representativos (p.e. Titulo 1, Titulo 2, etc...)
- Las entidades no tendrán imágenes.
- No se aplicará ningún tipo de maquetación (sólo HTML plano).
- No se considerarán usuarios (se asume que a los datos puede acceder el usuario no registrado).

Por tanto, será necesario implementar, al menos, los siguientes aspectos de la aplicación web:

- Servidor (Backend)
    - Entidad principal (con los atributos esenciales)
    - Carga de datos de ejemplo de la entidad principal en la base de datos.
    - API REST que permite la lectura de los recursos de la entidad principal.
        - Se utilizará OpenAPI para la documentación de la API REST. La documentación se alojará en la carpeta `/docs/api` en la raíz del repositorio y estará compuesta por:
            - El archivo de especificación OpenAPI: `api-docs.yaml`
            - Un archivo HTML generado a partir de dicha especificación: `api-docs.html`
- Cliente (Frontend)
    - Página principal que muestra los recursos cargados del servidor (mediante su API REST)

Como esta fase sienta las bases técnicas y de calidad para el desarrollo de las siguientes fases, se indican aquí todos los aspectos que deberían tenerse en cuenta:

4.2 Pruebas automáticas

Se deberán implementar pruebas automáticas de sistema (*end to end*, E2E) de la funcionalidad mínima desarrollada. En concreto, se deberán implementar las siguientes pruebas de sistema:

- Prueba de sistema del servidor:
  - Se probará la API REST
  - Se deberá implementar una prueba que verifique que los datos de ejemplo de la entidad principal se recuperan en la API REST.
  - Preferentemente se implementarán en Java usando Rest Assured. Aunque el alumno podría usar otras tecnologías.
- Prueba de sistema del cliente:
  - Se probará el interfaz de usuario:
  - Se deberá implementar al menos una prueba que verifique que los datos de ejemplo de la entidad principal se muestran en la página principal.
  - Preferentemente se implementarán en Java con JUnit usando Selenium. Aunque el alumno podría usar otras tecnologías.

Si se ha seleccionado la parte optativa "Pruebas automáticas unitarias y de integración", se deberá implementar, además de las pruebas previas:

- Pruebas unitarias:
  - Servidor: Prueba de la funcionalidad de los servicios con un doble de la base de datos.
  - Cliente: Prueba de la funcionalidad del componente con un doble de los servicios y un DOM virtual.
- Pruebas de integración:
  - Servidor: Prueba de la funcionalidad de los servicios usando la base de datos real.
  - Cliente: Prueba de la funcionalidad de los servicios conectando con la API REST real.

A partir de esta fase y a lo largo de todo el desarrollo, las pruebas se deberán ir ampliando, para que todas las funcionalidades tengan al menos una prueba de cada tipo.

Se deberán configurar las herramientas de pruebas automáticas para que midan la cobertura de los tests.

4.3 Integración Continua

Se debe añadir CI en la aplicación utilizando GitHub Actions (u otra tecnología alternativa) para que se ejecuten controles de calidad en las diferentes fases de madurez de código:

- Control de calidad básico:
    - Se ejecutará en cada commit que se haga en una de las ramas de funcionalidad.
    - Servidor:
        - Se compilará
        - Se ejecutarán los tests unitarios (si existen)
    - Cliente:
        - Se construirá
        - Se ejecutarán los tests unitarios (si existen)
- Control de calidad completo:
    - Se ejecutará cuando se intente mezclar el código de una rama de funcionalidad en la rama main.
    - Deberá configurarse de forma que, si los controles de calidad no pasan, no se pueda realizar la mezcla.
    - Se ejecutarán los siguientes tipos de tests:
        - Servidor:
            - Unitarios (si existen)
            - De integración (si existen)
            - De sistema
        - Cliente:
            - Unitarios (si existen)
            - De integración (si existen)
            - De sistema

Para desarrollar estas tareas automáticas se puede seguir el siguiente proceso:

- Se crea una rama “add-ci-workflow”
- Se añaden los workflows necesarios
- Se configurará su ejecución usando el *trigger* “workflow_dispatch” y *“push”* para facilitar el proceso de prueba y error.
- Cuando esté funcionando correctamente, se cambiarán los disparadores para que sean ejecutados cuando sea pertienente en cada caso.
- Finalmente, se mergeará la rama “add-ci-workflow” a main.

Si se ha seleccionado la parte optativa “Análisis estático de código” deberá quedar correctamente configurado en el sistema de integración continua en esta fase.

4.4 Documentación

Al finalizar esta fase la documentación de la fase 1 se dividirá en varios ficheros (que se guardarán en la carpeta docs) para facilitar su lectura.

Se describirán en la documentación los aspectos realizados hasta ese momento en el proyecto. Si sobre una sección concreta todavía no hay realizado nada, no se incluirá en la documentación de esta fase, si no en las posteriores.

La documentación ahora se estructurará de la siguiente forma:

- **Página Principal (README.md):**
  - **Título: Nombre de la aplicación web**.
  - Un párrafo que resuma la funcionalidad que tendrá la aplicación web
  - Algunos bocetos de pantalla que ilustren la funcionalidad
  - Se debe indicar claramente que sólo se han definido los objetivos funcionales y los objetivos técnicos de la aplicación y que se ha iniciado el desarrollo, pero que la implementación no es funcional todavía.
  - Índice con enlaces al resto de secciones de la documentación
- **Objetivos [no cambia]**
- **Metodología [no cambia]**
- **Funcionalidades detalladas [no cambia]**
- **Análisis [no cambia]**
- **Seguimiento [no cambia]**
- **Autores [no cambia]**
- **Guía de desarrollo:**
  - [Como tiene muchos apartados, al principio de la página habrá un link para cada uno de estos apartados a modo de índice. Si se considera necesario, las secciones con mucho contenido se pueden extraer a documentos independientes.]
  - **Introducción**
    - Párrafo indicando la arquitectura de despliegue de la aplicación web. Habitualmente se especificará que se trata de una aplicación web con arquitectura SPA (y habrá que explicar en qué consiste) y las partes que tiene (cliente, servidor y base de datos). Si es una aplicación distribuida se hará una descripción de alto nivel de este aspecto.
    - Resumen de esta descripción en formato tabla/lista:
      - Tipo: web MVC, Web SPA, API REST, microservicios…
      - Tecnologías: lenguajes, librerías, servicios adicionales.
      - Herramientas: IDEs empleados y herramientas auxiliares.
      - Control de calidad: Qué controles de calidad se aplican y qué tecnologías/herramientas se usan.
      - Despliegue: Cómo se empaqueta, distribuye y despliega la aplicación (Docker, Kubernetes). Entorno de despliegue (si aplica en este TFG).

- Proceso de desarrollo: iterativa e incremental, git, DevOps (CI/CD).

- **Tecnologías:** Tecnologías que usa la aplicación para su ejecución (no las herramientas usadas para su desarrollo).
  - Sólo se profundizará si se consideran poco conocidas. Si no, serán de un párrafo.
  - Si no es evidente se indicará para qué se usan en el proyecto.
  - En todas se indicará la URL oficial.

- **Herramientas:** IDEs empleados y herramientas auxiliares.
  - Sólo se profundizará si se consideran poco conocidas. Si no, serán de un párrafo.
  - Si no es evidente se indicará para qué se usan en el proyecto.
  - En todas se indicará la URL oficial.

- **Arquitectura:**
  - Despliegue: Arquitectura de despliegue (indicando procesos independientes y protocolos de comunicación)
  - API REST: Se añadirá un enlace que muestre la documentación de OpenAPI (como se menciona anteriormente) convertida a HTML. Para ello se utilizará el servicio (https://raw.githack.com).

- **Control de calidad:** Descripción de los controles de calidad que se han realizado.
  - Descripción de las pruebas automáticas de cliente y servidor:
    - Tipos de pruebas
    - Descripción de qué funcionalidades se prueban. Ya que están numeradas en el anexo, conviene que haya algún tipo de trazabilidad entre la prueba y la funcionalidad.
    - Estadísticas de las pruebas (número, cobertura, etc.). Mostrar captura de pantalla de su ejecución.
  - Herramientas de análisis estático de código (si se ha usado):
    - Captura de pantalla de los resultados de los análisis de código finales o a lo largo del tiempo.
    - Métricas del tamaño del código (número de clases, número de líneas de código, separadas por tecnologías...)

- **Proceso de desarrollo:** Descripción de los aspectos técnicos del proceso de desarrollo.

Proceso iterativo e incremental, que sigue los principios del manifiesto ágil y se apoya en algunas de las buenas prácticas de Programación Extrema (XP) y Kanban. No se puede decir que se ha aplicado Scrum, porque no se aplica.

- Gestión de tareas: GitHub Issues, GitHub Projects y gestión visual (tablero).
- Git: Se describe que se ha usado un repositorio git y la estrategia de ramas utilizada. Métricas de uso de git: Número de commits, número de ramas, etc.
- Integración continua: Se indican las tareas que realizan los flujos automáticos del sistema de integración continua (workflows de GitHub Actions).

- **Ejecución y edición de código:** Instrucciones de ejecución de la aplicación partiendo del código del repositorio:
  - Se indicará cómo clonar el repositorio.
  - Ejecución
    - Se indicará cómo ejecutar la base de datos (y otros servicios si fueran necesarios) para que pueda ejecutarse el servidor.
    - Se indicarán los comandos necesarios para ejecutar todas las partes de la aplicación.
    - Se indicará cómo acceder a la página web ejecutada en local.
  - Uso de herramientas
    - Se deberá dar una mínima explicación de cómo se usan las herramientas para desarrollar la aplicación (uso de entornos de desarrollo, herramientas auxiliares...).
    - Entre otras cosas se especificará cómo usar la herramienta para interactuar con la API REST del servidor (postman o similar). Se deberá proporcionar el fichero con los ejemplos de uso de la API REST (colección de postman o similar). Este documento deberá incluir ejemplos de peticiones a todas las operaciones de la API REST con datos de ejemplo.
  - Ejecución de tests
  - Creación de una release

4.5 Material de apoyo

- HTML y CSS: web-html-css

- JavaScript: javascript (Introducción y Estructuras de datos) y javascript-front
- Spring: spring
- Angular: angular
- Devops: devops (Integración (CI) y Entrega (CD) Continua)
- Pruebas: pruebas (Pruebas e2e de UI con Selenium y Pruebas API REST)
- Calidad software: calidad y mantenimiento

5 Fase 3 - Versión 0.1 - Funcionalidad básica y Docker

En esta fase se publicará la primera versión de la aplicación. Aunque no cuente con todas las funcionalidades definidas al comienzo del proyecto, ya debería podría ser de utilidad para los usuarios. Se podría considerar que esta versión es el MVP[8] (*Minimum Viable Product*).

A nivel técnico, en el desarrollo de esta versión se deben consolidar los aspectos técnicos más importantes del proyecto, que se describen a continuación.

5.1 Backend

El backend de la aplicación deberá tener las siguientes características:

- **Usuarios:** La aplicación gestionará los usuarios usando Spring Security. Si un usuario intenta acceder a una URL sobre la que no tiene permisos, se debería mostrar un error de acceso. Por ejemplo, si un usuario intenta editar un registro del que no es dueño, se deberá mostrar un error.
- **Comunicación segura:** La aplicación web deberá servirse por HTTPS en el puerto 443.
- **Imágenes en base de datos:** Para facilitar el despliegue de la aplicación en entornos restringidos, las imágenes se guardarán en la base de datos en vez de en el sistema de ficheros.
    - Si se opta por un despliegue en la nube, se podrán guardar las imágenes en Minio, un servicio de almacenamiento de ficheros que dispone de una imagen Docker para poder lanzarse de manera sencilla. Deberá utilizarse el SDK de AWS S3 (que es compatible con Minio), de manera que la migración del sistema a la nube (usando AWS S3) sea sencilla y pueda conservarse Minio para el desarrollo local y la realización de pruebas.
- **Arquitectura software:** Se deberán seguir las buenas prácticas de arquitectura Spring, separando la capa de controladores, lógica de negocio y de acceso a los datos.

[8] https://es.wikipedia.org/wiki/Producto_viable_m%C3%ADnimo

La API REST debe cumplir con las buenas prácticas. Algunas de ellas, son:

- Todas las URLs de la API REST comenzarán con "/api/v1".
- Los métodos http GET, PUT, POST, DELETE deben usarse de forma adecuada para las operaciones de consulta, modificación, creación y borrado.
- Las URLs deben identificar los recursos. Cada tipo de recurso se identificará en inglés y en plural.
- Los códigos de estado de respuesta deben usarse de forma adecuada.
- Las operaciones de creación deberán devolver el *header "Location"* cuyo valor es la URL con la que se puede obtener la representación del recurso recién creado.
- Las operaciones de la API REST que permitan filtrar o buscar elementos deberán tener los criterios de filtrado o búsqueda como parámetros de la URL (después de la interrogación "?").
- La API REST deberá ofrecer los listados de forma paginada.

Otras cuestiones:

- **Datos de ejemplo:** Se cargarán datos de ejemplo representativos en la base de datos al ejecutar la aplicación. Los datos e imágenes deberán ser representativos del tipo de aplicación web que se esté desarrollando: libros, restaurantes, productos, etc. Esta información se puede obtener de otras webs.

5.2 Frontend

El frontend de la aplicación deberá tener las siguientes características:

- **Interfaz de usuario:** En la medida de lo posible el interfaz de usuario se implementará usando librerías de componentes de alto nivel. Por ejemplo, ng-bootstrap[9] o angular-material[10].
- **Arquitectura software:** Se deberán seguir las buenas prácticas de arquitectura Angular, separando la lógica de gestión del interfaz en componentes y la lógica de conexión con el backend en servicios.
- **Páginas de error:** Cuando se intente acceder a una URL inexistente o se produzca un error en el servidor se deberá generar una página de error que tenga el mismo estilo gráfico que el resto de las páginas de la aplicación.
- **Paginación:** Todas las páginas que potencialmente vayan a mostrar más de 10 elementos deberán mostrar únicamente los 10 primeros y se permitirá al usuario cargar más. Para la carga de más elementos, se mostrará un botón o enlace de "Más resultados".

---

[9] https://material.angular.io/

[10] https://docs.github.com/es/get-started/using-github/github-flow

5.3 Controles de calidad

A medida que se vaya implementando nueva funcionalidad se ampliarán las pruebas automáticas para verificar que se comporta como se espera. Entre otras ventajas, esto permite evitar regresiones a medida que se añade nueva funcionalidad.

Si algún control de calidad (prueba automática o validación mediante análisis estático de código) no pasa en el entorno de CI, se debería solucionar el problema antes de continuar con el desarrollo.

Por otro lado, se exigirá una mínima calidad del código fuente. Al menos deberán tenerse en cuenta los siguientes aspectos:

- El código deberá estar formateado correctamente y usar las mismas reglas de estilo en todos los ficheros (se recomienda configurar el entorno de desarrollo para formatear al guardar y así evitar problemas de este tipo).
- El código y los comentarios de este deberán estar escritos completamente en inglés. Sólo podrán existir términos en castellano en el código cuando vayan a mostrarse en el interfaz de usuario. La existencia de texto en castellano en el código llevará aparejada una penalización en la calificación.
- Las variables, parámetros, atributos, clases e interfaces deberán tener nombres adecuados que describan su objetivo.
- Se evitará el código duplicado. Cuando dos fragmentos de código sean similares, se utilizarán las técnicas adecuadas para fomentar la reutilización (herencia, composición, subprogramación, etc).
- Los métodos no serán muy largos y no tendrán mucha complejidad ciclomática.
- El código deberá ser razonablemente eficiente. Por ejemplo, no se considerará válido hacer una consulta a la base de datos para obtener una lista de elementos y luego filtrar esa lista en memoria usando código Java.
- Si se quiere mostrar texto de depuración en la consola del proceso Java, se usará la librería de logs en vez de usar *"System.out.println()"*.
- El código debería estar correctamente estructurado de forma que los módulos no estén acoplados entre sí. Por ejemplo, para que la lógica de negocio no esté acoplada a los controladores, se deberá implementar un *@Service* con la lógica de negocio que será usado tanto por el *@RestController*. Este *@Service* será el que utilice los repositorios. Un controlador nunca podrá acceder a un repositorio directamente.

5.4 Empaquetado con Docker y Docker Compose

La aplicación web se empaquetará en una imagen docker que se publicará en DockerHub (en una cuenta del alumno).

Para ello, se creará una imagen Docker con las siguientes características:

- **Backend:** Ofrecerá la API REST en el puerto 443 por HTTPS
- **Frontend:** La aplicación Angular se publicará como un recurso estático del backend Spring de forma que esté disponible en la ruta https://localhost/

Para ello se usará un fichero Dockerfile.

Como la aplicación requiere de una base de datos, se ejecutará usando docker compose:

- Para el contenedor de la aplicación, se usará la imagen de la aplicación web.
- Para el contenedor de la base de datos, se usará la imagen de MySQL estándar de DockerHub.
- Para que la aplicación web se inicie correctamente una vez que la base de datos ha arrancado se utilizará el mecanismo de espera basado en *healthcheck*.
- La configuración de la base de datos y de la aplicación web se realizará utilizando variables de entorno.

El repositorio de código contendrá dos ficheros docker compose:

- docker-compose.yml:
  - Será el fichero para ejecutar la última versión publicada de la aplicación.
  - La imagen de la aplicación apuntará al tag "0.1" del registry en DockerHub.
- docker-compose-dev.yml:
  - Será el fichero para ejecutar la última versión de desarrollo de la aplicación (que puede ser inestable).
  - La imagen de la aplicación apuntará al tag "dev" del registry en DockerHub.

Para simplificar la ejecución de la aplicación con todas sus dependencias, el fichero docker-compose.yml se publicará como un artefacto OCI[11] con el tag "0.1" en el registry DockerHub.

Tanto el fichero "*Dockerfile"* como los ficheros "*docker-compose.yml"* se deben guardar en una carpeta *"docker" en el proyecto*.

5.5 Entrega continua y publicación de versiones

Se implementará un sistema de entrega continua con las siguientes características:

Los objetivos son los siguientes:

1. Cada vez haya un cambio en la rama main (merge de una rama de desarrollo) se generará una imagen docker con el tag "dev" y se publique en DockerHub. Se

[11] https://docs.docker.com/compose/how-tos/oci-artifact/

publicará en DockerHub el fichero “docker-compose.yml” como artefacto OCI con el tag “dev”.

2. Cuando haya una release GitHub (trigger “release”) se generarán una imagen docker y un compose con el tag como la <versión> de la release y se publicarán en DockerHub. Además, también actualizarán los tags “latest”.
3. Cuando el desarrollador quiera (trigger “workflow_dispatch”), se podrá generar una “build”, es decir, una imagen docker y un compose partiendo de cualquier rama (main o de funcionalidad) y en cualquier commit (último o uno concreto) y se publicarán en DockerHub. El tag será <nombre-rama>-<fecha-hora>-<commit>.

El alumno deberá crear uno o varios workflows para realizar estas tareas, pero es muy importante evitar la duplicación de lógica[12] en varios jobs.

Para que el código de repositorio en el tag git quede relacionado con la versión publicada, la versión del pom.xml, del package.json y del docker-compose.yml deberán ser igual a la versión del tag. Para conseguir esto, antes y después de realizar una release GitHub, se deberán hacer dos cambios en la rama main:

- Justo antes de hacer la release GitHub:
  - Cambiar en “pom.xml” del servidor a la versión a “0.1.0”
  - Cambiar en “package.json” del cliente a la versión “0.1.0”
  - Cambiar en el docker-compose.yml el tag de la imagen docker a “0.1”
- Justo después de hacer la release GitHub:
  - Cambiar en “pom.xml” del servidor a la versión a “0.2.0”
  - Cambiar en “package.json” del cliente a la versión “0.2.0”
  - Cambiar en el docker-compose.yml el tag de la imagen docker a “0.2”

Siguiendo estas instrucciones, al terminar la Fase 4, el estado de los artefactos del proyecto debe ser el siguiente:

- En GitHub
  - Debe existir una release de GitHub y un tag llamado 0.1 con las siguientes versiones en el código:
    - El pom.xml del backend con la versión 0.1.0
    - El package.json del frontend con la versión 0.1.0
    - El fichero docker-compose.yml apuntando a la imagen docker con tag 0.1
  - En la rama main deben existir las siguientes versiones:
    - En el pom.xml del backend la versión será 0.2.0-SNAPSHOT

[12] https://docs.github.com/en/actions/concepts/workflows-and-actions/avoiding-duplication

        - En el package.json del frontend la versión será 0.2.0
- En DockerHub
    - Debe existir una imagen con el tag “0.1.0” (con el código del tag 0.1 del repositorio)
    - Debe existir una imagen con el tag “latest” (con el código del tag 0.1 del repositorio)

Para que el alumno aprenda a realizar todos los pasos correctamente se recomienda que realice antes una o varias releases con números de versión 0.0.1, 0.0.2, 0.0.3, etc.. Una vez que tenga el proceso controlado, que haga la release 0.1.0.

5.6 Documentación

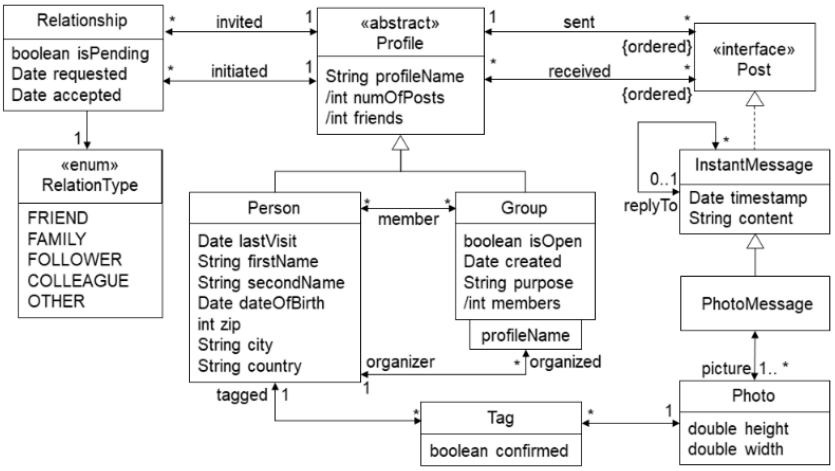
Se deberán incluir nuevas secciones y actualizar las secciones existentes.

Al finalizar esta fase la documentación quedaría de la siguiente forma:

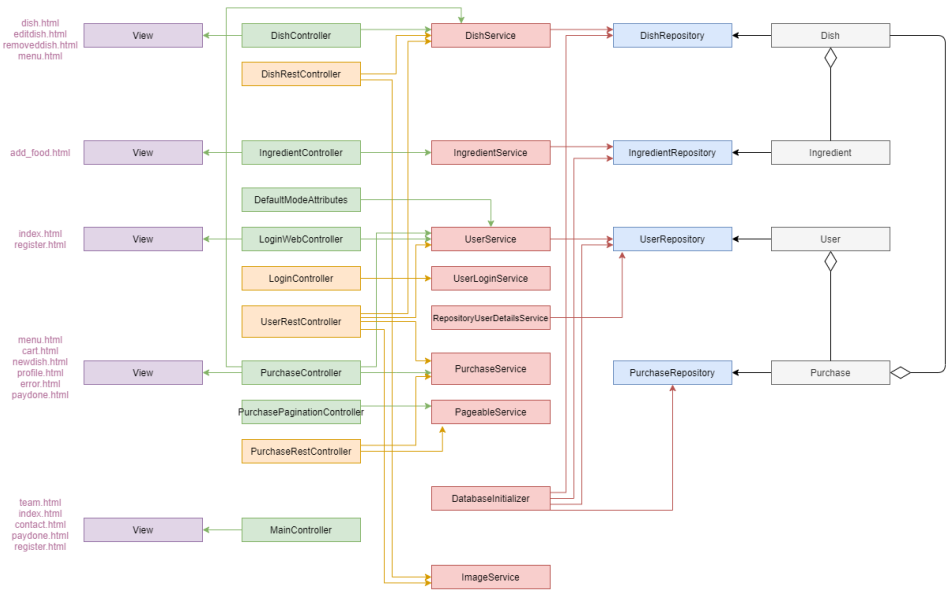
- **Página Principal (README.md):**
    - **Título: Nombre de la aplicación web**.
    - Un párrafo que resuma la funcionalidad que tiene la aplicación en la versión 0.1
    - Algunas capturas de pantalla de la versión 0.1.
    - Se debe indicar que se sigue desarrollando la aplicación para mejorar sus funcionalidades.
    - Vídeo de la versión 0.1 de 1 minuto mostrando las funcionalidades principales de la aplicación. Se mostrarán las funcionalidades separadas por tipos de usuarios (con voz en off explicando)
    - Párrafo resumiendo las funcionalidades que se espera incluir en las siguientes versiones.
    - Índice con enlaces al resto de secciones de la documentación
- **Funcionalidades:**
    - Se describirán las funcionalidades de la versión 0.1 ilustradas en el vídeo, pero con capturas de pantalla y una breve descripción de estas.
- **Funcionalidades detalladas**
    - Se actualiza la lista de funcionalidades indicando las que ya están implementadas y las que todavía no.
    - Se ajusta la descripción de las funcionalidades implementadas para que describan el comportamiento implementado.
- **Ejecución:** Instrucciones de ejecución de la aplicación partiendo del docker compose publicado en DockerHub.

- Se indicará que es necesario Docker Desktop en Windows y Mac y Docker y Docker compose en linux y links a las páginas con información sobre cómo se instalan.
- Se indicará cómo acceder a la aplicación web (credenciales de acceso de los datos de ejemplo)
- Se describirán los datos de ejemplo de la aplicación.

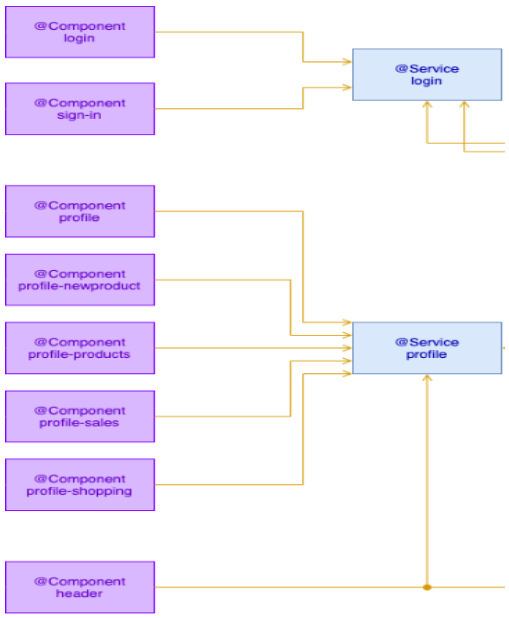
- **Guía de desarrollo:**
  - Se actualizará el contenido para que refleje el estado actual del proyecto
  - **Introducción [actualizado]**
  - **Tecnologías [actualizado]**
  - **Herramientas [actualizado]**
  - **Arquitectura**
    - **Modelo del dominio:** Entidades persistentes de la aplicación, sus atributos y sus relaciones. Un ejemplo de diagrama sería este:



    - **API REST [actualizada]**
    - **Arquitectura del servidor:** Diagrama de clases del servidor reflejando su separación por capas. Responsabilidades de cada capa (Controladores, servicios, repositorios...). No se incluirán ni atributos ni métodos en las clases.



- **Arquitectura del cliente:** Diagrama de clases del cliente reflejando su separación por capas. Responsabilidades de cada capa (Componentes, servicios...).



- **Control de Calidad [actualizado]**
- **Despliegue:** Cómo se realiza el empaquetado y distribución
  - Empaquetado y distribución: una única imagen Docker para cliente y servidor, docker compose para coordinar, uso de Kubernetes, serverless...). Se indicará la URL para acceder al artefacto de la aplicación (DockerHub)
- **Proceso de desarrollo [actualizado]**
  - Gestión de tareas [actualizado]

        - Git [actualizado]
        - Integración y entrega continua [actualizado con información sobre la entrega continua]
        - Versionado: Descripción del procedimiento de lanzamiento de versiones (*releases*), fechas en las que se ha publicado cada versión, descripción de alto nivel de las funcionalidades que incluye, etc.
    - Ejecución y edición de código [actualizado]
- **Seguimiento [no cambia]**
- **Inicio del proyecto**
    - **Objetivos [no cambia]**
    - **Metodología [no cambia]**
    - **Funcionalidades iniciales (Sección "Funcionalidades detalladas" de la fase anterior)**
    - **Análisis [no cambia]**
- **Autores [no cambia]**

6 Fase 4 - Versión 0.2 - Funcionalidad intermedia

En esta fase se implementarán las funcionalidades definidas como intermedias. No se requieren grandes cambios en la arquitectura ni en el proceso de desarrollo.

Cuando se hayan implementado se actualizará la documentación, el vídeo y se publicará la release 0.2.0.

6.1 Despliegue

Esta release deberá quedar desplegada en un entorno diferente al entorno de desarrollo.

Si no se ha seleccionado ninguna parte opcional relacionada con el despliegue, la aplicación deberá desplegarse en una máquina (propia, de la universidad o creada en un hosting o proveedor de computación en la nube), conectado por SSH, y con la base de datos desplegada con Docker compose y con el volumen local para que no se borren los datos en cada actualización.

Si se ha seleccionado alguna parte opcional relacionada con el despliegue, se deberá desplegar tal y como se ha seleccionado.

Se aprovechará para publicar un post en el blog de medium.

6.2 Documentación

La documentación deberá actualizarse para reflejar las nuevas funcionalidades implementadas.

Además, deberá actualizarse con información sobre despliegue y si se ha implementado, también de despliegue continuo. Las secciones afectadas son las siguientes:

- ∉ Guía de desarrollo > Despliegue
- ∉ Guía de desarrollo > Proceso de desarrollo > Despliegue continuo

7 Fase 5 - Versión 1.0 - Funcionalidad avanzada

Esta fase es similar a la fase 5 pero con las funcionalidades avanzadas.

Una vez finalizado el TFG se archivará el repositorio, indicando que el desarrollo ha concluido.

7.1 Documentación

La documentación deberá actualizarse para reflejar las nuevas funcionalidades implementadas.

Se actualizarán todos aquellos aspectos que hayan evolucionado o cambiado en esta última fase.

8 Fase 6: Memoria

En esta fase, se elaborará el primer borrador de la memoria del Trabajo de Fin de Grado. Este documento debe escribirse siguiendo las indicaciones de Guía para el desarrollo de la memoria del TFG v2.

9 Fase 7: Defensa

La fase final del proyecto se centra en la preparación y realización del acto de defensa del Trabajo de Fin de Grado (TFG). Este acto se preparará siguiendo las indicaciones de la Guía para la preparación de la defensa del TFG.

10 TFGs para alumnos con doble titulación

En caso de que un alumno esté interesado en un TFG de tipo web y esté cursando algunas como titulación doble GII+GIC o GII+GIS podrá realizar los dos TFGs de tipo web, pero adaptados de la siguiente forma:

El primer TFG se implementará la aplicación web con las siguientes partes optativas:

- Pruebas automáticas unitarias y de integración
- Análisis estático de código

En el segundo TFG se desarrollará con las siguientes partes optativas:

- Despliegue:

    - Despliegue continuo
    - Despliegue con Kubernetes en la nube
    - Despliegue integrado con servicios cloud (S3, RDS, Cloud Formation…)
- Arquitectura:
    - Dividir en 3 o más servicios
    - Utilizar tecnologías de comunicación complementarias
    - Escalabilidad y tolerancia a fallos

O con una propuesta concreta que realice el tutor.